KQL Field Manual

Hunting the Polyglot Poisoning intake ledger — keep this open

Everything you need to hunt this table, in one place. Keep it open in a second tab. Workshop: **Polyglot Poisoning: Hands-On Hunting for Linguistic Attacks in AI** — Sahara Al-Madi, Linguistic Security Institute.

1 · Reading a KQL query

A table goes in. Each | pipe hands the current result to the next step. You choose columns, calculate, expand and sort until only useful evidence remains.

```
TableName
| where   — filter before you filter again
| extend  — add a calculated column
| mv-expand — one row in, many rows out
| summarize — collapse rows into counts
| project — keep only the columns you want
| sort by — put the interesting thing on top
```

> After every pipe, ask two things: **what rows remain**, and **what does one row now represent?** Most mix-ups here come from losing track of the second one.

2 · The table

Every AI submission to this workspace is logged as one row in one custom table: `PolyglotPoisoning_CL`. Start here:

```
PolyglotPoisoning_CL
| take 20
```

COLUMN	MEANING	
`TimeGenerated`	when it arrived	HUNT THIS
`UserPrincipal`	who sent it	HUNT THIS
`Language`	which tongue, at dialect level	HUNT THIS
`Variant`	how the request was shaped	HUNT THIS
`RiskScore`	scored risk — a real number; the wire trips at 5	HUNT THIS
`Patterns`	detections it tripped — a CSV string, several per row	HUNT THIS
`PromptPreview`	first 200 characters of the request	READ LAST

> **Six of the seven are metadata.** Hunt those. Open `PromptPreview` only once the labels have cut the table to a handful of rows — then read the survivors.

3 · The codes

Query the codes, not the glosses: `Variant == "V2"`, never `"dialect"`.

Variant — what was done to it

`V1`	asked plainly — a standard, direct request
`V2`	asked in a regional variety or dialect

Only these two codes ever appear. Register, framing and script live in `Patterns`, not here — a request only framed as an old manual or a museum label stays `V1`.

Language — where it was asked

`ar-MSA`	Modern Standard Arabic
`ar-EG`	Egyptian Arabic
`ar-TN`	Tunisian Arabic
`es-ES / fr-FR / de-DE`	Spanish / French / German
`it-IT / pt-PT`	Italian / Portuguese
`bn-IN / hi-IN`	Bengali / Hindi
`sw-KE / th-TH`	Swahili / Thai
`en-US`	English

> **Arabic is not one language in this ledger.** Egyptian and Tunisian differ about as much as varieties of Spanish — different words, different grammar. A filter that handles one may not handle the other at all, which is exactly why they are separate codes. Filter the full code (`Language == "ar-TN"`), or use `startswith` for a family.

Language says WHERE a request was asked. Variant says WHAT was done to it. They vary independently — any dialect can carry any handling method. A crew that changes both at once is not four incidents; it is one campaign wearing four coats.

4 · The operators you need today

OPERATOR	WHAT IT DOES
`where`	keeps rows that match a condition — chain several, cheapest first
`project`	keeps only the columns you name, in that order
`summarize … by`	collapses rows into one row per group — `count()`, `dcount()`
`count()`	how many rows — `dcount(X)` is how many *distinct* X
`split(S, ",")`	cuts a CSV string into an array
`mv-expand`	one row with an array in, one row per element out
`extract(rx, n, S)`	pulls capture group *n* out of a string by regex
`sort by / order by`	orders the result — `asc` or `desc`
`startswith / endswith / contains`	substring tests; `has` is faster when you want a whole token

5 · The four moves

Move one — reduce, one condition at a time

Stack `where` clauses and watch the row count fall. Three conditions true at once is usually enough to isolate a single row.

```
PolyglotPoisoning_CL
| where Language == "ar-TN"
| where Variant == "V2"
| where RiskScore >= 6
```

Move two — count, and watch a row change meaning

`summarize` collapses rows into groups. After it, one row is no longer one submission — it is one *group*. Track that.

```
PolyglotPoisoning_CL
| summarize rows = count() by Language, Variant
| sort by rows asc
```

Move three — split before you count

`Patterns` packs several detections into one comma-separated string. Counting the column directly counts **combinations**, not detections. That approach stops short by design — expand the values first.

```
PolyglotPoisoning_CL
| mv-expand stamp = split(Patterns, ",") to typeof(string)
| summarize rows = count(), langs = dcount(Language) by stamp
| sort by rows asc
```

> **Rare is a shortlist, not a verdict.** Do not sort ascending and grab the bottom row. Ask which value keeps company with too many different languages and too many different combinations — that is the one that was stamped on, not the one that is just uncommon.

Move four — look under the wire

The detection fires above a threshold. The interesting population is the band underneath it: scored, flagged, and let through.

```
PolyglotPoisoning_CL
| where RiskScore >= 2 and RiskScore < 5
| where Language != "en-US"   // this practice slice only
| project Language, Variant, RiskScore, Patterns
```

And then — read the survivors

Only now open the text. Pull fenced fragments out of prose with a regex:

```
| extend piece = extract(@"\[([^\]]+)\]", 1, PromptPreview)
| where isnotempty(piece)
```

6 · Traps and habits

- **Scope your time range** before anything else. An unscoped query over a wide window is slow and noisy.
- **`has` beats `contains`** when you want a whole token — it is faster and it will not match inside another word.
- **String comparison is case-sensitive by default.** Use `=~` for case-insensitive equality.
- **After `summarize`, re-read your column names.** A column not in the `by` clause or an aggregate is gone.
- **Exclude the leaderboard's own traffic:** `| where UserPrincipal !endswith "@lb"`
- **Don't trust a board — run the query.** Every number in this workshop is reproducible from the table in front of you. Reproduce one.

> **Take this back to work.** These seven columns are close to the minimum an AI gateway should log if you ever want to hunt it: who asked, in which dialect, shaped how, scored what, flagged by which detections, and a preview to pivot on. Most telemetry carries none of the middle four. If your logs cannot answer *which dialect is failing*, that is the gap.

License

© 2026 Sahara Al-Madi, Linguistic Security Institute. Released under Creative Commons Attribution-NonCommercial 4.0 — use it, teach from it, fork it, adapt it for your own team. Full license and attribution terms are in the **Participant Instructions**, under *License & attribution*. Commercial use: `linguisticsecurityinstitute@gmail.com`